Regarding kernel launch (launching the code) o

```
++

void code
```

Kernel Execution in a nutshell

```
__host__
void vecAdd(...)
{
  dim3 DimGrid(ceil(n/256.0),1,1);
  dim3 DimBlock(256,1,1,);
  vecAddKernel<<<DimGrid, DimBlock>>>
  (d_A,d_B,d_C,n);
}


__global__
void vecAddKernel(float *A, float *B, ...
```

More on CUDA function declarations

Function	Executed on the:	Only callable from the:
`__device__ float DeviceFunc()`	Device	Device
`__global__ void kernelFunc()`	Device	Host
`__host__ float HostFunc()`	Host	Host

Compiling CUDA program

NVCC compiler -

Multi-Dimension Kernel Configuration (for assignment)

- we can have 2D and 3D blocks and grids
- Processing 2D grid is good for pictures

– Lets say you divide a screen into a 16x16 grids, it won't always perfectly fit inside our picture, so we can have some threads that are idle, but it is easier to write code for 2D grid than 1D grid

Example Kernel code for PictureKernel

```
 // Scales every pixel value by 2.0
__global__ void PictureKernel(float *d_Pin, float* d_Pout, int height, int width){

  // Calculate row # of the d_Pin and d_Pout element
  int Row = blockIdx.y * blockDim.y + threadIdx.y;

  // Calculate column # of the d_Pin and d_Pout element
  int Col = blockIdx.x * blockDim.x + threadIdx.x;

  // each thread computes one element of d_Pout if in range
  if((Row < height) && (Col < width)){
    int idx = Row * width + Col;
    d_Pout[Row*width + Col] = 2.0*d_Pin[Row*width + Col]; // used for row major order
  }
}
```

Host Code for Launching PictureKernel

```
// Assume picture m n,
// m pixels in y dimension and n pixels in x dimension
// input d_Pin has been allocated
dim3 DimGrid((n-1)/16 + 1, (m-1)/16 + 1, 1); // m is height, n is width
dim3 DimBlock(16, 16, 1);
PictureKernel<<<DimGrid, DimBlock>>>(d_Pin, d_Pout, m, n);
```

Colors to GrayScale conversion

- Color (3 channels) to Grayscale (1 channel)
- You can do a formula, where you multply each channel by

`gray pixel[I,J] = 0.21*r + 0.71*g + 0.07*b`, defined by rec709 – Essentially a dot product of `<r,g,b>` and `<0.21, 0.71, 0.07>`

```
#define CHANNELS 3
__global__ void Color2GrayKernel(unsigned char * grayImage, unsigned char *
colorImage, int height, int width){
  int x = threadIdx.x + blockIdx.x * blockDim.x;
  int y = threadIdx.y + blockIdx.y * blockDim.y;

  if(x < width && y < height){
    int grayOffset = y * width + x;
    int rgbOffset = grayOffset * CHANNELS;
    unsigned char r = colorImage[rgbOffset];
    unsigned char g = colorImage[rgbOffset + 1];
    unsigned char b = colorImage[rgbOffset + 2];
    grayImage[grayOffset] = 0.21f * r + 0.71f * g + 0.07f * b;

  }
}
```

Show this grayscale image converter in action for extra credit

- MUST BE IN CUDAC

Thread Block Scheduling

Objective:

- Learn how CUDA thread scheduling works

Transparent Scalability:

- Each block can excute in any order relative to others
- Hardware is free to assign blocks to any processor at any time

– Kernel scales to any number of parallel processors

SM: Streaming Multiprocessors

Threads are assigned to SM in block granularity, and each block is assigned to one SM. Each SM has a fixed number of resources (registers, shared memory) that are shared among the blocks assigned to it. The number of blocks that can be assigned to an SM is limited by the resources required by each block and the total resources available on the SM.

Ex with Fermi arch:

- Up to 8 blocks to each SM
- Fermi SM has 1536 threads

– could be 256 `(threads/blocks)*6` blocks or 512 `(threads/blocks)*3 blocks`

What block size should I use??

- 2D algs should I use 8x8, 16x16, or 32x32 blocks blocks for Fermi?
- Limitation: 1536 threads/SM & 8 blocks/SM

Block Size	Threads/Block	Max Block/SM	num of blocks in SM	num of threads utilized in SM
`8x8`	64	8	8	512/1536
`16x16`	256	6	1536	1536/1536
`32x32`	1024	1	1024	1024/1536

- When it comes ot choosing block size, we want to maximize the number of threads per block, but we also want to have enough blocks to keep all the SMs busy.

So for Fermi, 16x16 blocks are a good choice because they allow us to fully utilize the SMs while still providing enough blocks to keep them busy.

- You need to optimize it yourself, this is part of compile time optimization, and you can use tools like NVIDIA Visual Profiler to help you find the optimal block size for your specific application.

Von Neumann Model:
- CPU goes through instructions and it tells the processor what to do

– CPU goes through register files, goes into ALU, then stores the result back into register files, and then goes to the next instruction – One processor unit, memory, a single control unit (PC/IR) then I/O

Warps as scheduling units:
- Each block is executed as 32-thread warps
- Warps are scheudling units within SMs
- Implementation decision, not part of CUDA programming model
- Threads in a warp execute in SIMD

Warp example:
- 3 blocks assinged to an SM and each block has 256 threads
- How many warps are in each block?

Warps in Multi-dimensional thread blocks:

- Thread blocks are first linearized into 1D in row-major order
- Linearized thread blocks are partitioned into warps
- Partitioning scheme is consistent across devices

– Exact size of warps may change from generation to generation, but the partitioning scheme will be the same

Warp Divergence:
- Pushing threads into different paths SMs are SIMD Processors

– Control unt for instruction fetch, decode, and control is shared among multiple threads

SIMT Execution Model
- Challenge: How to handle branch operations when different threads in a warp follow a different path through program?
- Solution: Serialize different paths
- This means to make good pipelines utilizations
- You don't want to have idle threads, so you want to minimize the number of divergent paths within a warp
- Happens to threads in a warp when they have different control warp paths, such as when they execute different instructions or access different memory addresses

Divergence examples:
- When branch or loop condition is a function of thread indices
- Example kernel statement with divergence:

```
if(threadIdx.x > 2){
  // do something
} else {
  // do something else
}
```

– This creates different control paths for threads in a block – Decision granularity < warp size; thread 0, 1, and 2 follow different paths than the rest of the threads in the first warp

- Example without divergence:

```
if(blockIdx.x > 2){
  // do something
} else {
  // do something else
}
```

– Decision granularity > warp size; all threads in the first few warps follow the same path, and all threads in the remaining warps follow the same path, so no divergence within warps – We are not breaking control paths within warps, so we have good pipeline utilization

- In the example of vector addition kernel

```
int i = threadIdx.x + blockIdx.x * blockDim.x;
if(i < n) C[i] = A[i] + B[i];
```

- control divergence isn't an issue when that diverging path is actually empty and doesn't

stall out any other threads

Analysis of 1000 elements in a vector

- Assume 256 threads, 8 warps in each block
- All threads in Block 0,1,and 2 are within valid range
- Most warps in block 3 will not have control divergencve
- one warp in block 3 will have control divergence
- Effect of serialization on control divergence

W2 Thursday lecture

GPU Instruction Set Architecture (ISA)

- NVIDIA defines a virtual ISA, called PTX (Parallel Thread Execution)

Generatic GPU architecture

- Lots of SIMT Core Clusters (Single Multiple Threads)
- Each cluster has multiple SMs (Streaming Multiprocessors)
- Each SM has multiple CUDA cores, which are the actual processing units that execute instructions
- Each SM has its own shared memory and registers, which are used to store data for the threads that are executing on that SM
- Each SM can execute multiple warps of threads concurrently,
- SIMT Core CLusters go through an interconnect network to access the Memory partition. Then off-chip DRAM (GDDR5 for modern GPUs) is accessed through memory controllers.

– The off-chip DRAM is seen as the global memory by the threads executing on the SMs.

Inside SIMT Core

- SIMT front end/ SIMD backend
- Fine-grained multithreading to hide latency

– Interleave warp execution hides latency – Register values of all threads stay in core

SIMT Front end

- Fetches and decodes instructions
- Schedules warps for execution
- A scoreboard to track data dependencies and resource conflicts
- Contains two schedulers, I-Cahche and Issue unit, and a warp scheduler

SIMD Datapath

- Operand Collector takes value from SIMT front end and sends it to the appropriate execution unit
- ALUs execute instructions for all threads in a warp
- MEM units execute memory instructions for all threads in a warp
- Scheduler 3 is from the Operand Collector to the ALUs and MEM units, and it schedules instructions for execution based on the availability of operands and resources

How the SIMT Hardware Stack Works

- There are multiple warps and it looks like a state machine in order of execution going through paths
- SIMT = SIMD Execution of Scalar Threads

Following slides are relevant to the next assignment

Parallel Computing Patterns (Reduction)

- How CUDA organizes their memory hierarchy and how to optimize memory access patterns for better performance
- We understand there is global memory, every SIMT Core has memory subsystem, shared memory, texture cash, constant (variable) cache, and registers

Declaring CUDA Variables

Variable Declaration	Memory	Scope	Lifetime
`int LocalVar;`	Register	Thread	Thread
`__deviice__ __shared__ int SharedVar;`	Shared	Block	Block
`__device__ float d_var`	Global Memory	Device	Kernel Execution
`__shared__ float s_var`	Shared Memory	Block	Kernel Execution
`float r_var`	Register	Thread	Kernel Execution
`__constant__ float c_var`	Constant Memory	Device	Program Execution

Example: Shared Memory Variable Declaration

```
void blurKernel(...){
  __shared__ float s_data[256];
  // rest of the kernel code
}
```

Shared Memory in CUDA

- Copy everything you can on shared memory for fast memory accesses
- One in each SM
- Declared per block, shared among threads in the same block
- Need to be shared into Global data or else it will be uninitialized and will not have the correct values when accessed by threads in the block
- (Think of this memory as being the stack memory when it comes to functions in C++)

Tiling/Blocking - Basic Idea

- Do a block copy from global memory to shared memory (do the first 4 blocks, then shift to the next 4 blocks, etc.)

– 4 being an arbitrary number, but as much as you can fit into shared memory

“Partition and Summarize”

- Partition the data into smaller chuncks that can fit into shared memory, and then summarize the results from each chunk to get the final result

– Example: Parallel reduction for summing an array of numbers – In ML Traiing, reduced ML training are useful for distillation

Reduction Enables other techniques

- Used to clean up after commonly used parallelizing transformations (used for ML training)
- Privatization (Optimizing for reduction)

– Multiple threads write into an output location – Replicate output location so each thread has private otuput location (privatization) – Then use reduction tree to combine the results

What is reduction computation

Summarize a set of input values into one value using a “reduction operation”

- Max, min, sum, product, logical and/or, etc.

Often used with user defined reduction operation functions as long as the operation

- Associative and commutative (order of operations doesn’t matter)

Refuction in Sequential Algorithm

- Find max of a sequential array,

– Do a for loop and find what is largest

- We can use parallel reduction tree algorithm to perform N-1 operations in log(N) steps

Work Efficiency Analysis of Parallel Reduction

- How many operations do we perform?
- For N input values, the reduction tree performs

– (1/2)N + (1/4)N + (1/8)N + … + 1 = N-1 operations

Basic Reduction Program

How are we going to implement this reduction in CUDA?

Parallel Sum Reduction

- Parallel implementation

– Each thread adds two values in each step – Recurisvely halve # of threads – Takes log(n) steps for n elements

- The lecture example shows that we have N threads, then half the threads each time.

We show that at the end, we are only using 1 thread for the final result and have warp divergence. – We are stalling some threads – This is the Naive thread to Data Mapping

Reduction steps

```
for (unsigned int stride = 1; stride <= blockDim.x; stride *= 2){
  if (threadIdx.x % (2*stride) == 0){
    sdata[threadIdx.x] += sdata[threadIdx.x + stride];
  }
  __syncthreads();
  if(t % stride == 0){
    partialSum[2*t] += partialSum[2*t + stride];
  }
}
```

- This makes sure that the even threads are the ones executing the reduction after the first step (first reduction)
- Then the final calculation is taken by the left most threads

– Thread 0 will have the final result after log(n) steps, but we have a lot of idle threads and warp divergence

Thread Index Usage Metrics

- We want smart thread indexing
- Shift all the threads down

– Some algorithms can shift the index usage to improve divergence behavior

Keep active threads consecutive (keep them physically close to each other)

- Always compact the partial sums into the front locations in teh partialSum[] array
- Here, our block begins at stride = blockDim, then keeps getting halved until we get to 1, and we are using the first half of the threads each time, so we are keeping the active threads together and minimizing divergence

Quick Analysis

For 1024 thread blocks

- No divergence in the first 6 steps

– 1024, 512, 256, 128, 64, 32 consecutive threads are active in each step – All threads in each warp either all acgive or all inactive

- Final 5 steps will still have divergence, but only 1 thread is active in the final step, so we are not stalling out many threads at the end